

Final Project Proposal: CUDA-Accelerated CNN Inference Engine from Scratch

Ashish Dasu — CS5330 Pattern Recognition & Computer Vision — Prof. Bruce Maxwell

Motivation

I've used PyTorch throughout this course without any concrete understanding of what `model.to("cuda")` actually does. Convolution and pooling are familiar as mathematical operations. What I don't know is how they become GPU code running on thousands of parallel threads. I want to find out by writing those kernels myself, so that by the end I can read a CUDA kernel and know what each thread is doing.

Project Overview

I will implement a forward-pass CNN inference engine in C++ and CUDA, without relying on PyTorch, cuDNN, or any deep learning framework. The engine will run the LeNet-style network trained on MNIST from Project 5, replacing every layer operation with hand-written CUDA kernels that execute directly on the GPU.

Technical Approach

Weights will be exported from the Project 5 trained PyTorch model as raw binary floats, transferred to the GPU via `cudaMemcpy`, and the full forward pass runs with no Python or framework dependency at runtime. The following components will be implemented in CUDA C++:

- **Convolution kernel:** naive implementation first for correctness, then optimized with tiled shared memory to reduce global memory traffic. Prior work shows that coalesced memory access in convolution kernels yields significant speedups over naive implementations [1].
- **GEMM kernel:** for fully connected layers, implemented as a tiled general matrix multiply that loads data into fast shared memory once and reuses it across all threads in the block [2].
- **Elementwise kernels:** ReLU and softmax, with each GPU thread handling one output element in parallel.
- **Max pooling kernel:** each thread computes one output value by scanning its corresponding input window.

Scope. Minimum viable: correct forward pass with naive kernels matching PyTorch outputs. Target: tiled shared-memory optimization for convolution and GEMM. Stretch: meaningful speedup toward cuDNN on convolution throughput. The project is submittable at the minimum level even if optimization milestones slip.

Evaluation

Correctness will be validated by comparing output logits against the PyTorch reference on the MNIST test set. Performance will be measured with NVIDIA Nsight Compute, using memory bandwidth utilization and kernel throughput as primary metrics. Results will be compared against cuDNN as an upper-bound reference [3].

Risks

The main implementation risks are softmax numerical stability (the naive formulation overflows on large logits and needs the max-subtraction trick), bank conflicts and boundary handling in the shared-memory convolution, and padding conventions that must match PyTorch exactly. Each kernel will be built as a standalone unit test against a hand-computed reference before integration, so bugs surface locally rather than as end-to-end mismatches.

Connection to Course Material

Project 5 involved building, training, and analyzing a CNN on MNIST. This project takes the trained network from that work and asks what it takes to run it without PyTorch. The course covered convolution and pooling as mathematical operations; this project implements them as parallel programs. It is the same pipeline, one layer deeper.

Tools and Environment

NVIDIA CUDA Toolkit, C++17, CMake, NVIDIA Nsight Compute, Python (weight export only). Development will use Google Colab (NVIDIA T4 GPU) [4].

References

- [1] Jordà, M., Valero-Lara, P., & Peña, A.J. *cuConv: CUDA Implementation of Convolution for CNN Inference*. Cluster Computing, 25, 1459–1473, 2022. <https://doi.org/10.1007/s10586-021-03494-y>
- [2] Boehm, S. *How to Optimize a CUDA Matmul Kernel for cuBLAS-like Performance: a Worklog*. 2022. <https://siboehm.com/articles/22/CUDA-MMM>
- [3] NVIDIA Corporation. *CUDA Deep Neural Network Library (cuDNN)*. <https://developer.nvidia.com/cudnn>
- [4] NVIDIA Corporation. *CUDA C++ Programming Guide*. <https://docs.nvidia.com/cuda/cuda-c-programming-guide/>